

STATIC CODE ANALYSIS REPORT

SonarQube Quality & Security Assessment

Project: Cohort 9 MERN Notes App

Stack: MERN (MongoDB, Express, React, Node.js)

Analysis Engine: SonarQube Community Build v26.8.0.126808

Version 1.0.0 | Analysis Date: August 22, 2026

Prepared by:Ali hussnain

1. Introduction

This report presents the results of a static code analysis conducted on the Cohort 9 MERN Notes App using SonarQube Community Build. The objective of this analysis is to evaluate the overall code quality, security posture, reliability, and maintainability of the application prior to release. SonarQube was configured to scan both the backend (Node.js/Express) and frontend (React) source trees, covering JavaScript and CSS files across the project.

The sections that follow walk through the analysis in the same order a reviewer would naturally examine it in the SonarQube dashboard: starting from the project listing, moving into the Overview and Quality Gate results, then drilling down into complexity, security ratings, per-file security review, module-level code breakdown, configured quality profiles, and finally the project configuration details.

2. Project Dashboard

The SonarQube "Projects" page is the entry point of the analysis platform. It lists every project registered on the server along with a condensed health summary for each, allowing a reviewer to quickly gauge project status without opening it. For this instance, only one project — Cohort 9 MERN Notes App — is registered.

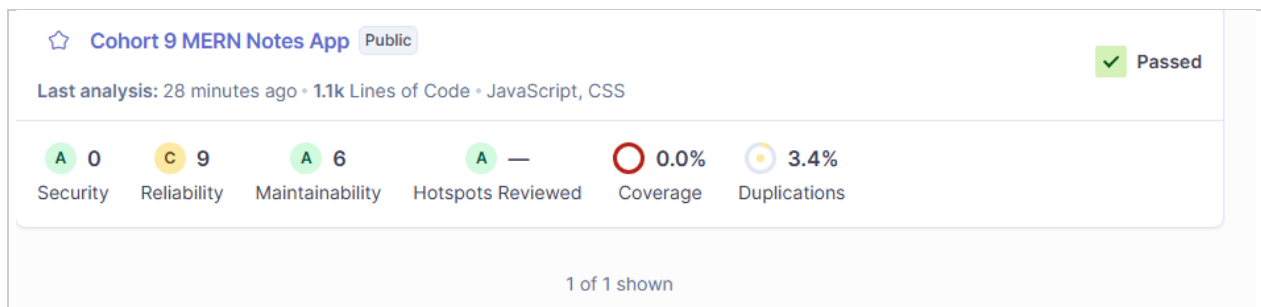


Figure 1: Project summary card on the Projects dashboard, showing Quality Gate status and top-level ratings.

The project card confirms that the last analysis passed the Quality Gate, comprises roughly 1.1k lines of code written in JavaScript and CSS, and carries an overall Security rating of A and Maintainability rating of A. The Reliability rating of C is the one area that stands out and is examined in detail in Section 5.

The left-hand panel of the Projects page also exposes filter facets that summarize the same metrics in aggregate form, which is useful once multiple projects are tracked on the same server:



Figure 2: Project Size facet — the application falls in the 1k–10k lines-of-code bracket.

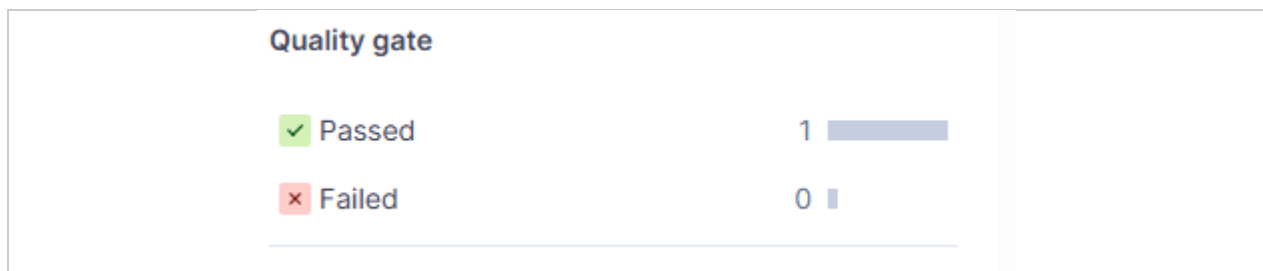


Figure 3: Quality Gate facet — 1 of 1 projects currently passing.



Figure 4: Security Review facet — 100% of security hotspots have been reviewed (Rating A).

3. Overview and Quality Gate

Opening the project leads to its Overview page, which is the primary landing screen for day-to-day monitoring. It surfaces the Quality Gate verdict first, since this is the pass/fail signal that determines whether a build is fit for release.

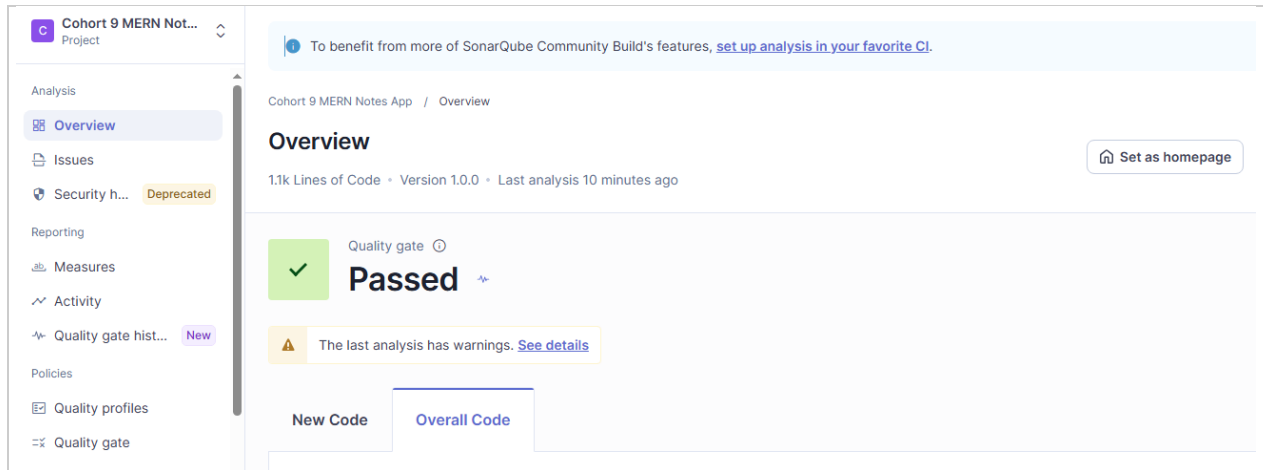


Figure 5: Project Overview header confirming the Quality Gate has Passed, with a warning noting that the last analysis produced advisory warnings.

Below the Quality Gate banner, the Overview page breaks the result down by the six core measures SonarQube tracks: Security, Reliability, Maintainability, Coverage, and Duplications, plus the count of accepted (deliberately unresolved) issues.

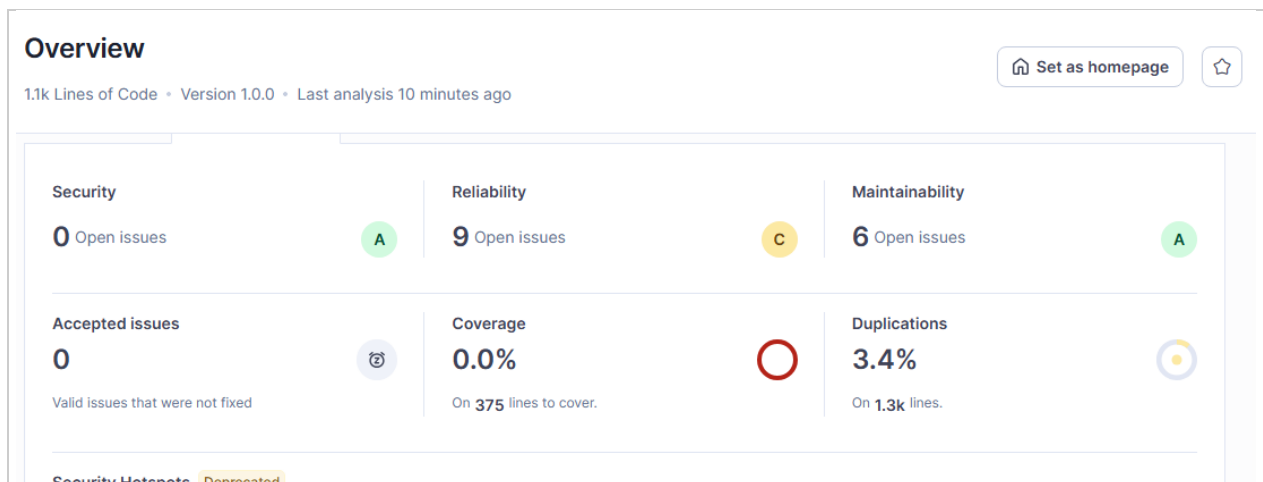


Figure 6: Overview measures panel — Security (A, 0 issues), Reliability (C, 9 issues), Maintainability (A, 6 issues), test Coverage (0.0% of 375 lines), and Duplications (3.4% of 1.3k lines).

Two figures merit attention here. First, test Coverage is reported at 0.0%, meaning the analysis run did not ingest any test-execution coverage reports — this is a configuration gap rather than a statement that no tests exist, and it should be corrected by wiring a coverage reporter (e.g. Istanbul/nyc for the Node/React stack) into the CI pipeline before the next analysis. Second, Reliability sits at a C rating with 9 open issues, making it the weakest of the three rating categories and the main area to prioritize for remediation.

The Activity view confirms this was the first recorded analysis for the project and that it passed cleanly against the default "Sonar way" quality gate, with the initial run logging 13 issues, 0.0% coverage, and 3.4% duplication:

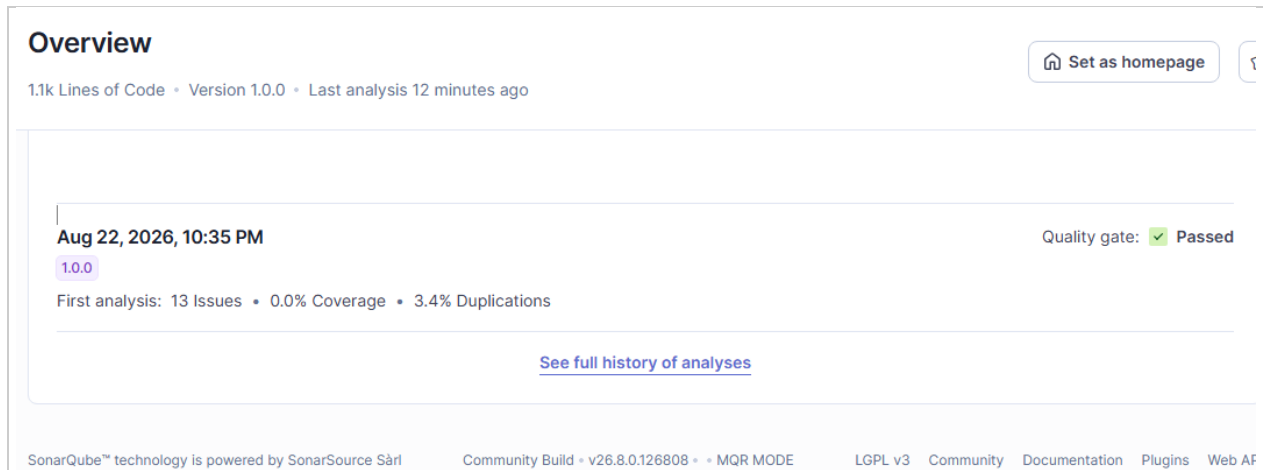


Figure 7: Analysis history — first analysis on August 22, 2026 (version 1.0.0), Quality Gate Passed.

The Quality Gate History view provides the same confirmation from a release-safety angle: of the one release made under this project, 100% were "safe" (i.e., shipped while the gate was green), and 0% were risky.

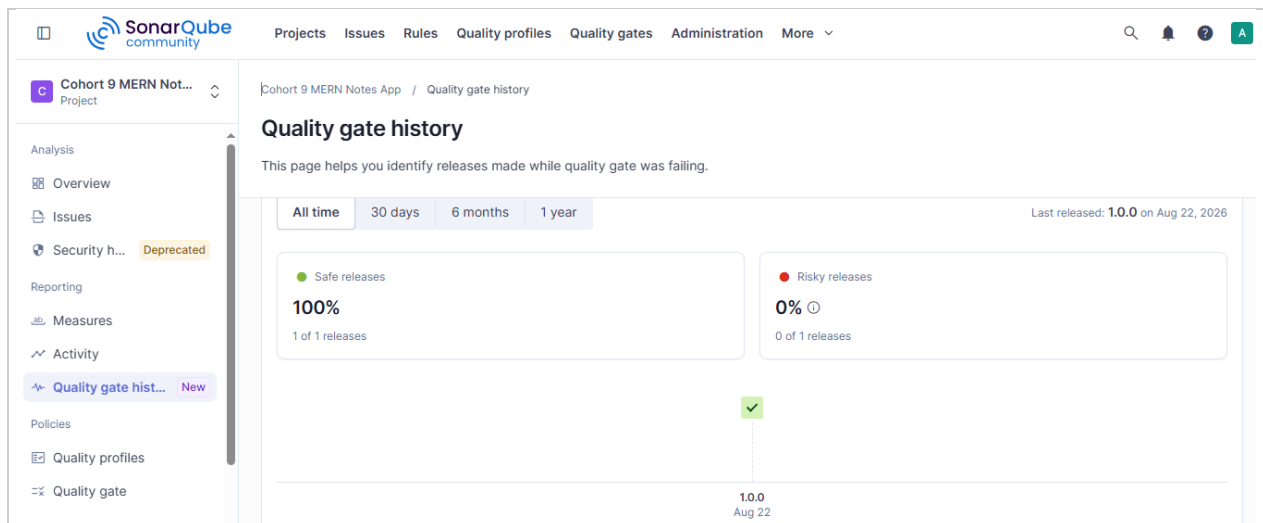


Figure 8: Quality Gate history — 100% of releases (1 of 1) were made while the gate was passing.

4. Code Complexity

The Measures tab exposes deeper structural metrics beyond the top-level ratings. Cyclomatic Complexity across the codebase totals 150, with Cognitive Complexity at 91 — both are moderate for a project of this size (roughly 1,119 lines of code) and are split fairly evenly between the backend (90) and frontend/src (60) directories.

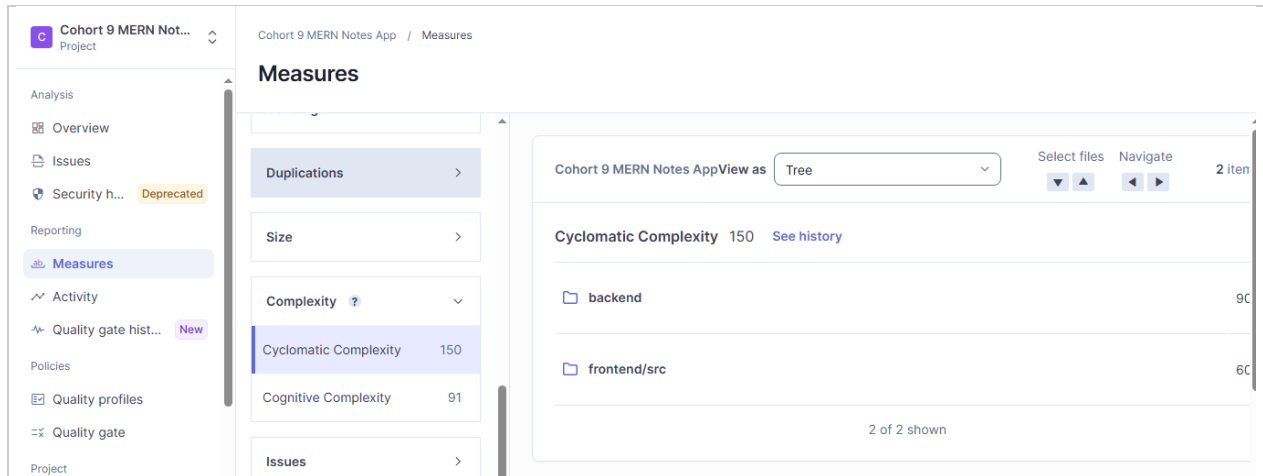


Figure 9: Complexity breakdown by directory — Cyclomatic Complexity of 150 split across backend and frontend/src.

No single directory dominates the complexity figures, which suggests logic is reasonably distributed rather than concentrated in one problematic module. As the codebase grows, it is worth revisiting the highest-complexity individual files (drillable from this same screen) to keep functions small and testable.

5. Security Analysis

Security is assessed through two complementary lenses in SonarQube: a Security Rating derived from vulnerability-type issues, and a Security Review measure derived from how thoroughly Security Hotspots have been triaged. Both currently sit at the best possible rating (A) for this project.

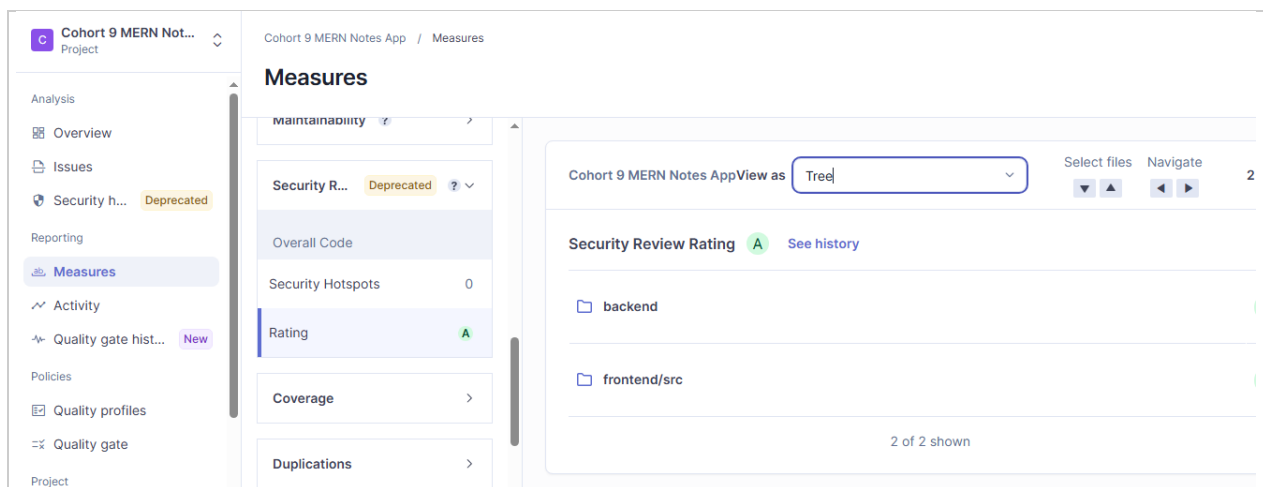


Figure 10: Security Rating (legacy "Security Hotspots" view) — Rating A with 0 unresolved security hotspots, split evenly across backend and frontend/src.

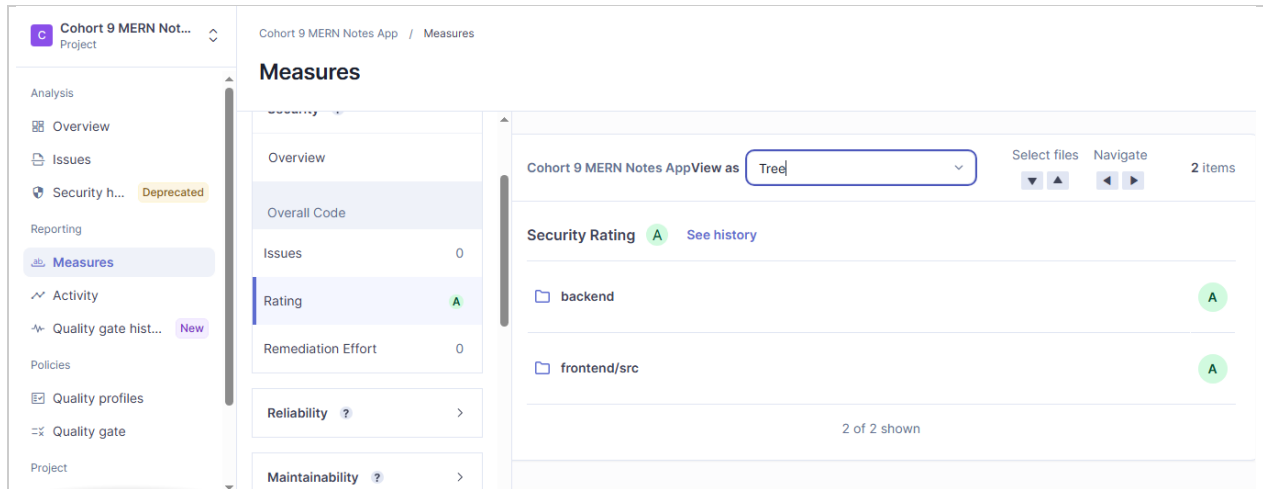


Figure 11: Security Rating detail — 0 open issues and 0 estimated remediation effort, Rating A for both backend and frontend/src.

The Risk overview chart plots every module by its worse-of Reliability/Security rating on the vertical and horizontal axes, with bubble size representing lines of code. Both tracked modules cluster at the safe (A/A) end of the chart, with the larger bubble corresponding to the larger frontend/src directory.

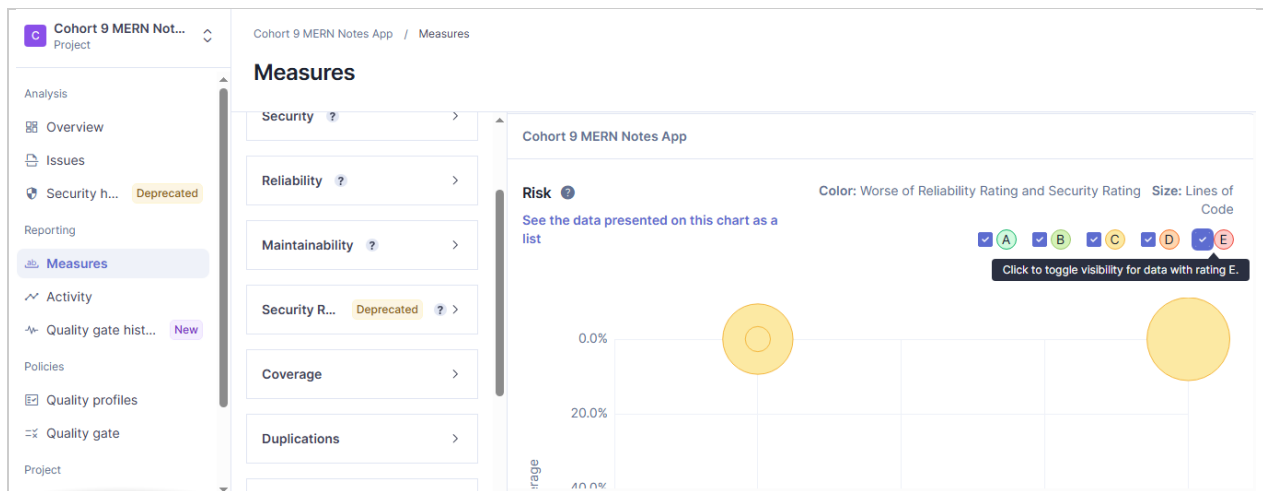


Figure 12: Risk matrix — both backend and frontend/src modules plot at the lowest-risk (Rating A) position.

Taken together, these views indicate that no outstanding vulnerabilities or unreviewed security hotspots exist in the codebase as of this analysis — a strong result, though it should be re-verified after each significant feature addition rather than treated as a one-time guarantee.

6. File-Level Security Rating

For full traceability, SonarQube also allows the Security Rating to be inspected per individual file rather than only at the directory level. All 22 analyzed files carry a Security Rating of A, confirming that the clean

result in Section 5 holds consistently across the codebase rather than being skewed by a few large, low-risk files. The complete file listing is reproduced below in the order it appears in SonarQube:

The screenshot shows the SonarQube interface for 'Cohort 9 MERN Notes AppView as'. A search bar contains 'List'. Navigation controls include 'Select files', 'Navigate', and a '22 items' indicator. The 'Security Rating' is 'A' with a 'See history' link. The file list shows:

File Path	Security Rating
frontend/src/App.css	A
frontend/src/App.jsx	A
backend/src/middleware/auth.js	A

Figure 13: File-level Security Rating, part 1 of 6 — App.css, App.jsx, middleware/auth.js.

The screenshot shows the continuation of the file list in SonarQube. The files and their security ratings are:

File Path	Security Rating
frontend/src/App.jsx	A
backend/src/middleware/auth.js	A
backend/tests/auth.test.js	A
backend/src/controllers/authController.js	A
backend/src/routes/authRoutes.js	A

Figure 14: File-level Security Rating, part 2 of 6 — App.jsx, middleware/auth.js, auth.test.js, authController.js, authRoutes.js.

 frontend/src/pages/ Dashboard.jsx	A
 backend/src/config/ db.js	A
 backend/src/middleware/ errorHandler.js	A
 frontend/src/ index.css	A
 backend/src/utlis/ jwt.js	A

Figure 15: File-level Security Rating, part 3 of 6 — *Dashboard.jsx*, *db.js*, *errorHandler.js*, *index.css*, *jwt.js*.

 backend/src/config/ logger.js	A
 frontend/src/components/auth/ Login.jsx	A
 frontend/src/ main.jsx	A
 backend/src/models/ Note.js	A
 backend/src/controllers/ noteController.js	A

Figure 16: File-level Security Rating, part 4 of 6 — *logger.js*, *Login.jsx*, *main.jsx*, *Note.js*, *noteController.js*.

 frontend/src/components/notes/NoteEditor.jsx	A
 backend/src/routes/noteRoutes.js	A
 backend/tests/notes.test.js	A
 frontend/src/pages/Profile.jsx	A
 frontend/src/components/auth/Signup.jsx	A

Figure 17: File-level Security Rating, part 5 of 6 — NoteEditor.jsx, noteRoutes.js, notes.test.js, Profile.jsx, Signup.jsx.

 frontend/src/components/auth/Signup.jsx	A
 backend/src/models/User.js	A

Figure 18: File-level Security Rating, part 6 of 6 — Signup.jsx, User.js.

Every file in the list — authentication middleware, controllers, models, routes, React components, and test files alike — retains the top A rating, with no exceptions found during this scan.

7. Code Structure and Module Breakdown

The Code explorer aggregates all six headline metrics per directory in a single table, giving a side-by-side comparison of the project's two main modules.

	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
Cohort 9 MERN Notes App	1,119	0	9	6	0	0.0%	3.4%
backend	442	0	3	2	0	0.0%	4.2%
frontend/src	677	0	6	4	0	0.0%	2.9%

Figure 19: Code breakdown — backend (442 lines, 3 reliability issues, 4.2% duplication) versus frontend/src (677 lines, 6 reliability issues, 2.9% duplication).

The frontend/src directory is the larger of the two by lines of code and by absolute issue count, which is expected given it hosts the bulk of the React UI components; however, its duplication percentage (2.9%) is lower than the backend's (4.2%), suggesting backend code has slightly more repeated logic that could be candidates for shared utility functions.

8. Quality Profiles

Each language analyzed in the project is checked against a Quality Profile — a curated rule set that determines which issues SonarQube flags. Both languages used in this project are currently assigned the instance's default "Sonar way" profile, a broadly-applicable rule set maintained by SonarSource.

Language	Current quality profile	Active Rules	Actions
CSS	Instance default: Sonar way Built-in	40	
JavaScript	Instance default: Sonar way Built-in	415	

Figure 20: Quality Profiles in use — CSS (40 active rules) and JavaScript (415 active rules), both on the built-in "Sonar way" profile.

Relying on the default profile is a reasonable starting point, though as the project matures the team may wish to tailor the JavaScript profile — for example, tightening rules around React hooks usage or Express error handling — to better reflect house coding standards.

9. Project Configuration

Finally, the Project Information page consolidates the administrative configuration of the scan: which quality gate and quality profiles are bound to the project, its unique project key, visibility, and available status badges for embedding in external documentation such as a README file.

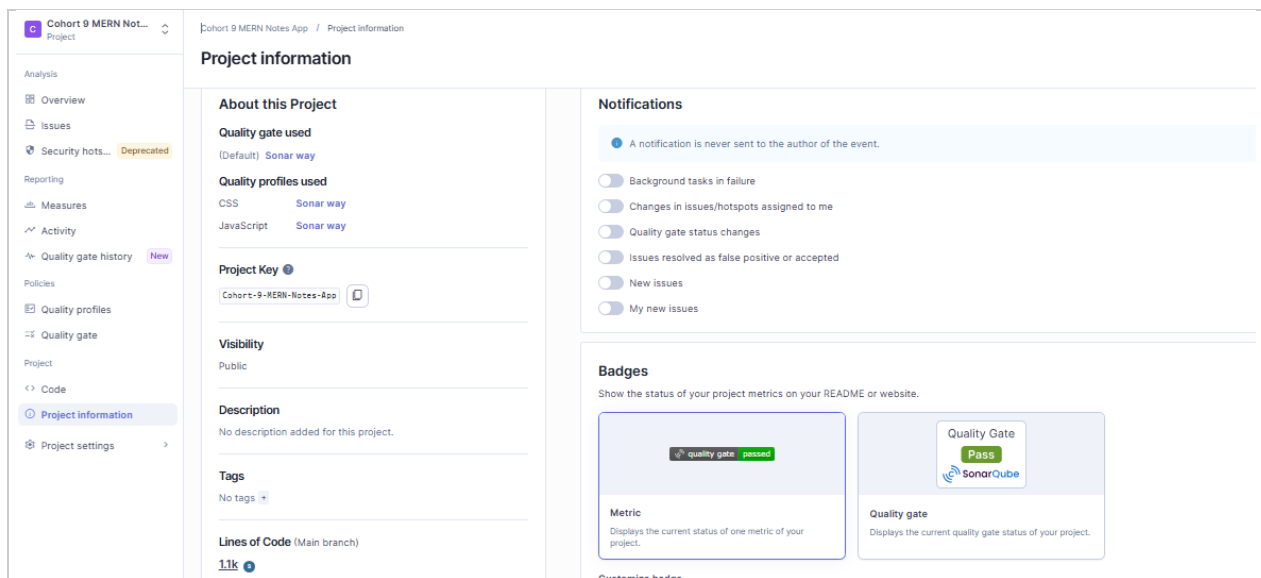


Figure 21: Project Information — Quality Gate "Sonar way", Quality Profiles for CSS and JavaScript, project key, visibility, and embeddable badges.

The project is registered under the key "Cohort-9-MERN-Notes-App", is set to Public visibility, and uses the default Sonar way quality gate — consistent with everything observed throughout this report.

10. Summary and Recommendations

Overall, the Cohort 9 MERN Notes App passes its Quality Gate with a clean Security rating (A), a clean Maintainability rating (A), and no unresolved security hotspots across all 22 scanned files. The following points summarize the key findings and recommended next steps:

- Reliability is rated C with 9 open issues — this is the top priority for remediation and should be reviewed first via the Issues tab.
- Test coverage is reported at 0.0%; this appears to reflect a missing coverage-report integration in the analysis pipeline rather than a true absence of tests, and should be fixed by feeding LCOV/Istanbul output into the next SonarQube scan.

- Code duplication stands at 3.4% overall (4.2% in backend, 2.9% in frontend), which is within a normal range but worth monitoring as the codebase grows.
- Security posture is strong: 0 open security issues and 100% of security hotspots reviewed, consistently across both backend and frontend modules.
- Quality profiles are currently the SonarQube defaults; customizing them to the team's standards is a reasonable future improvement but not an immediate concern.

With the Reliability issues addressed and coverage reporting wired into the pipeline, the project would be in a strong position to maintain a consistently passing Quality Gate through future releases.